LineVigil — Right-of-Way Protection Intelligence (One-page)

Purpose: manage excavation requests, patrol verification, pipeline telemetry and imagery detections.

Architecture

- **Backend:** Node.js + Express, Socket.IO for realtime.
- **Database:** PostgreSQL with PostGIS (spatial types & queries). Fallback: in-memory mock in `db.js`.
- **Frontend:** React (Vite), Tailwind CSS, Leaflet (react-leaflet) for maps.

Core Data Models

- `pipeline_routes` — LineString geometry; nearest-pipeline queries.
- `excavation_requests` — Point geometry, `distance_to_pipeline`, `status` and `risk_level`.
- `imagery_detections`, `patrol_tracks`, `verification_logs`, `cgd_ga_telemetry`.

Auth & Security

- Passwords hashed with *bcryptjs*. JWT tokens via *jsonwebtoken* with `JWT_SECRET`.
- `middleware/auth.js` verifies Bearer token and attaches `req.user`.

Key Flows (Stepwise)

1. Login → POST `/api/auth/login` → verify password → return JWT → store in `localStorage`.
2. Contractor creates excavation → backend computes nearest pipeline (PostGIS or mock) → store request with Point geometry.
3. Patrol fetches tasks → verifies incident → backend writes `verification_logs` and updates `excavation_requests`.

Tech Rationale

- PostGIS for accurate spatial queries; Leaflet + GeoJSON for map visualization.
- Socket.IO enables real-time patrol notifications and telemetry updates.
- Mock fallback supports offline demo/testing without PostgreSQL.

Run Summary (short)

Backend: install, optionally `npm run init-db`, then `npm run dev`. Frontend: `npm run dev` in `frontend`.

Notes: Demo fallbacks (client auth bypass & mock DB) exist for local testing and must be removed for production. Ensure `JWT_SECRET` and DB credentials are secured.

Generated: 2026-05-29 — LineVigil